Paper5 UID

why we arrange the words in certain order?
is it random? is it a habbit? or is it complex subconcious computation and optimisation happening

Smooth ride theory (Levy and Jaeger 2007)
Do we want to accelerate our car and then break and accelerate again or we want a smooth ride?
offcourse later

But this paper says we want a bumpy ride

Surprisal

less surprisal = less information
information = -ve log of conditional probability i.e. surprisal = $-\log P(\text{word} \mid \text{context or sentence})$
conditional probability = what is probability of a word appearing given two words before!
-ve log = probabilities ~~of~~ are very small number between 0 and 1
-ve log makes smaller number appear bigger and easy to work with.
UID theory suggests we have limited cognitive buffer.
~~Ther~~ So we try to keep information density constant.
In english (SVO) -> this smoothing is done by deleting the words like "that"
In Hindi (SOV) -> we test this smoothing by moving the words.

English e.g.:- [The boy] ate [the apple]
Subject verb. Object

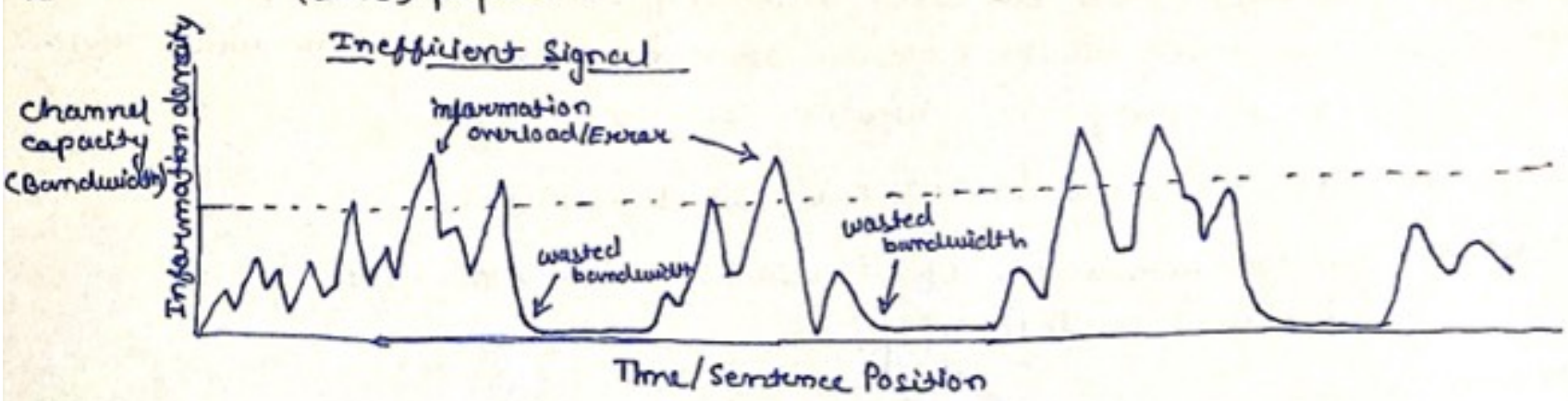
Hindi e.g.:- [Raam ne] seb [Khaaya]
Subject object verb.

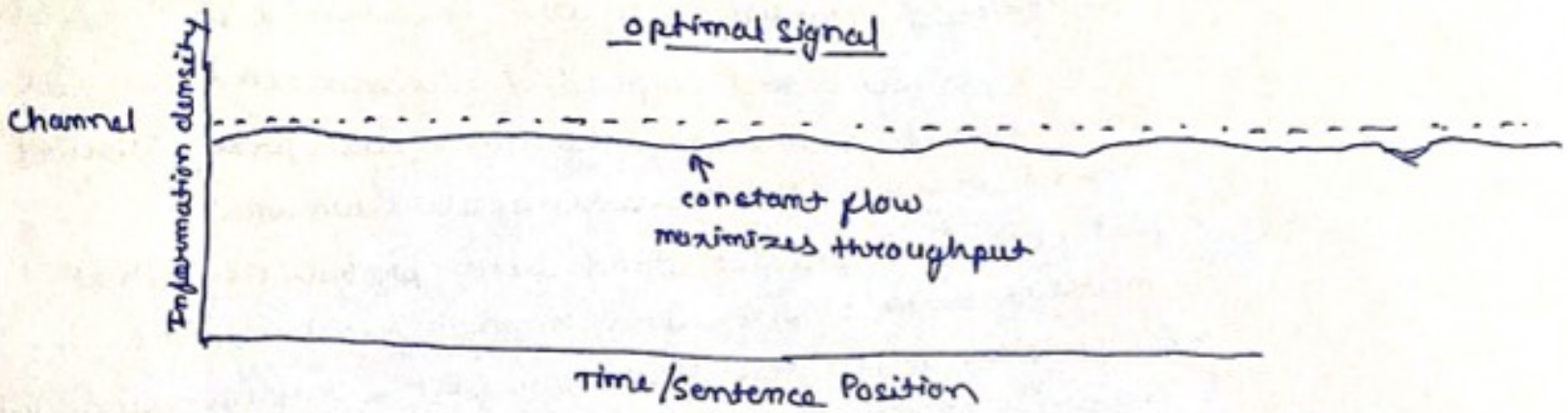
Subject - object can be moved in Hindi

Do Hindi speakers use this flexibility of word order to smooth information spikes?

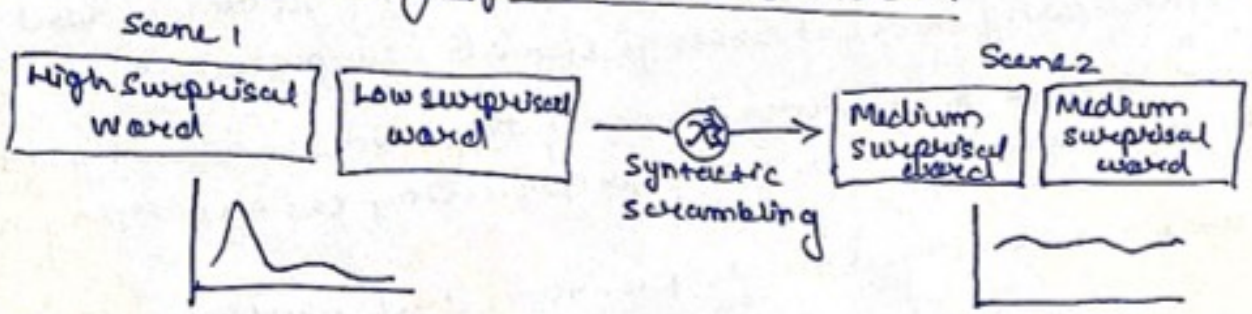
Paper 3

UID (2018) paper.





Mechanism: Smoothing signal via word order.

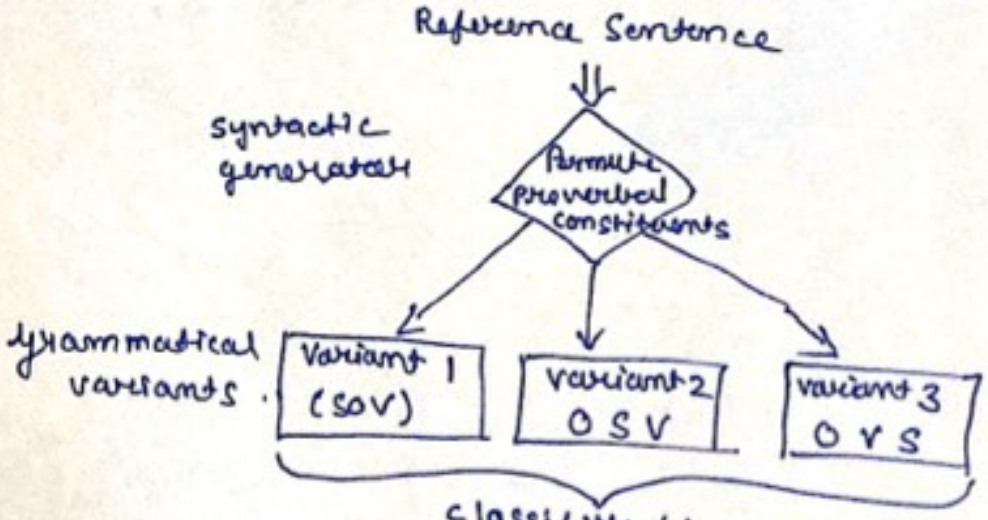


If a word is highly unexpected, UID predicts we should:

1. Move it later in sentence (allowing context to build up)
2. Insert functional words (like 'that') to dilute the density

To test this we need a language ~~rocket~~ with massive flexibility
we need Hindi SOV scrambling

Generating counterfactual.



classifier (logistic regression)
(Distinguish real vs fake based on smoothness)

Methodology -

- we cannot understand a choice unless we know what was rejected
- Dataset: - Hindi sentence vs English
  - variants (175,801) variants (8,385)

Hypothesis

- If UID is true, real sentence should be mathematically smoother than artificial variants

Test components Notebook.

checking individual components

what are token and sentences ?

Tokens :- one word in a sentence with all its information

Sentence :- A collection of tokens that form a complete thought

why we need these ?

Computers or LLMs read words as tokens to :-

1) understand their meaning
2) How each word is related to each other.
3) which word is main action verb.
4) which words are subjects, objects.

```
@dataclass
class Token:
    id: int  position in sentence
    form: str  actual word (राम)
    lemma: str  Base form (खाता)
    upos: str  Part of speech (Noun, verb)
    xpos: str  Language specific pattion-tag
    feats: dict  Features like gender, number
    head: int  which word this word dependion
    deprel: str  Relationship type.
```

In sentence रामु खाता है :-

रामु is token 1 खाता is token 2

```
get_root():
    gets main verb
get_children():
    Finds all words depend on specific word
get_word_sequence():
    gets just words in order
```

what is CoNLL-U format ?

It is standard of writing sentences with their grammatical information

Sample line : 1 राम राम propn NNP case=Nom 2 subj --

position :- 1
Word = ram
lemma = ram
Universal pos = PROPN (proper noun)
language pos = NNP
Features : case = Nom (nominative case)
Head = 2 (depends on word 2)
Relation = nsubj (nominal subj)

we need this format to read universal tree bank.

we will read its sentences and their grammer.

parse_feats(s)

Takes "case=Nom|Number=Sing" makes {"case": "Nom", "Number": "Sing"}

parse_conllu(filepath, max_sent=None):
#Reads the file line by line
#when it sees empty line -> sentence is complete.
#creates Token objects and adds them to a sentence

Lexical surprise

English:

Common words e.g. "the" -> low surprisal
Rare words e.g. "xylophone" -> high surprisal

Hindi:

है (is) appears more often -> low surprisal
~~कुरन्दोत्र~~ appears less often -> high surprisal

What is a trigram?

Trigram looks at 3 words together to predict next word

Example: Given "राम् रहता" what comes next?
- है (is) very likely
- गाता (sings) is very unlikely

Why trigrams?

To calculate probability of words in a sentence

Math (simplified)

surprisal = $-\log_2$(P(word | previous 2 words))
probability = 0.5 -> surprisal = 1 bit
probability = 0.25 -> surprisal = 2 bits
probability = 0.125 -> surprisal = 3 bits
Higher surprisal = more unexpected = more information

The Code Explained

```
class TrigramModel
  def train(self, sentences)
      #counts how often each word appears
      #counts how often pairs appears (bigrams)
      #counts how often triples appears (trigrams)
```

Smoothing :- I have never seen this exact combination, I will make a guess based on what I have seen.

UID Measures

Uniform information density = speakers spread information evenly across a sentence

e.g. spreading butter on bread

Why UID matters in sentence?

we want to test if Hindi speakers naturally arrange words to spread information evenly.

If this is true then sentences in dataset should be more uniform than variants.

UID Measures used:-

1. UIDglob (Global UID)

```
def UID_global(info):
    return variance of information.
```

Measures how much information varies across whole sentence

Negative variance = More uniform = higher UID_global

2. UIDloc (Local UID)

```
def UID_Local(info):
    returns -(average of (difference betn neighbouring words)^2
```

Measures how much information changes from word to word

smaller differences in values = more inform

like checking if each bite has similar butter to next bite

3. UIDglobNorm (Normalised Global)

```
def uid_global_norm(info):
    return -variance relative to mean
```

- Same as UIDglob but divides by the average
- Makes it compatible across sentences of different lengths

4. UIDlocNorm (Normalised Local)

```
def uid_local_norm(info):
    return -local differences relative to meanⁿ2
```

Same as UIDloc but normalised

Also makes it comparable across sentences.

5. UIDloc PrevNorm (Previous word Normalised)

```
def uid_local_prev_norm(info):
    return -differences relative to previous word (not mean)
```

compares each word to the word before it (not the average)

very local measure

we try all 5 measures and check which one captures "uniformity" best

Variant generator

Variants are different order of words in same sentence without changing meaning of sentence

E.g. राम सेब खाता है

variant 1 - सेब राम खाता है

We want to test if original sentence from dataset is more uniform than generated variants

get Preverbal Constituents

chunks of words before the verb that can be

```
def get_preverbal_constituents(sent)
    # Find all words that came before the main verb.
    # groups them by which word they depend on.
```

Preverbal constituents of राम सेब खाता है :-

राम - subject सेब - object खाता है - verb.	सेब राम खाता है

```
def generate_variants(sent, max_var=99):
    # gets all permutations of preverbal_chunks
    # generates new sentences with different order of preverbal constituents
```

Pairwise calculation (classifying between variants and real corpus sentences)

Pair 1: (variant, corpus) → label 1

Pair 2: (corpus, variant) → label 0.

```
def make_diff(f1, f2):
    substracts features of 1st from 2nd
    This tells direction of difference

def create_pairs(corp, var):
    creates both orderings
    (corpus, variant) and (variant, corpus)
    Labels them 0 and 1
```

if corpus surprisal = 10, UID = -5

variant surprisal = 15, UID = -3

then, Diff = 5, 2

Cross Validation

Test model on different parts of data to make sure it works

Divide sentence in words or 5 parts (folds)

check surprisal for each part

First train on 4 parts, check on remaining one part.

Do this for each part as test

Return average accuracy

```
def cross_validation (x, y, n_folds=5):
    Splits data into 5 parts
    Train on 4 parts, test on 1
    Repeats 5 times (each part gets a turn on test)
    Returns avg accuracy
```

Correlation Analysis

Correlation is measure of how much things move together

+1: when one goes up, other goes up.

-1: when one goes up, other goes down.

0: No relationship.

Why check correlation?

To make sure UID measures are not measuring same things for. surprisal or measuring something unique which can be used for classification.

```
def compute_correlation(x, y)
    # pearson correlation coefficient
    # tells us if x and y move together
```

Lexical Surprise (Trigram Model)

Math (Intuitive)

Surprisal(word) = $-\log_2$ (P(word | previous two words))

P(है | राम खाना) = 0.8 -> Surprisal = 0.32 bits (low surprisal, expected)

P(गाना | राम खाना = 0.01 -> surprisal = 6.64 bits (high surprisal, surprising)

Smoothing Explained

```
def get_prob (self, word, prev1, prev2):
    #60% from trigram (3 word history)
    #30% from bigram (2 word history)
    #10% from unigram (just word itself)
```

Because we haven't seen every word in Hindi we will need this function to calculate probability of unknown words.

Syntactic surprisal

Measures how "surprising" each word is based on grammar structure
Sometimes word can be predicted not just by previous words but by grammar. e.g. after a subject and object, you expect a verb.

Lexical surprisal	Syntactic surprisal
Based on word sequences	Based on grammar structure

```
def get_state (self, tokens, i):
    creates a "state" from pos tags and dependencies
    e.g. "PROPN:nsubj | verb: root"

def compute_surprisal (self, sent):
    # For each word, compares state probability to previous state.
    # High ratio change = High surprisal
```

UID Measures

1. UID glob (global variance)

```
def uid_global (info):
    m = mean (info)
    return -sum (x-m) **2 for x in info) / len (info)
```

Measures how much each word differs from average.

Sentence A: [3, 3, 3, 3] → Variance = 0 → UIDglob = 0 (perfectly uniform)
Sentence B: [1, 5, 1, 5] → Variance = 4 → UIDglob = -4 (not uniform)
Higher (less negative) = more uniform

2. UIDloc (Local difference)

```
def uid_local (info):
    diffs = [(info[i] - info[i-1]) ** 2 for i in range (1, len(info))]
    return -sum (diffs) / len (info)
```

Measures how much information jumps between consecutive words
Example:

[3, 3, 3, 3] → Differences = [0, 0, 0] → UIDloc = 0 (smooth)
[1, 5, 1, 5] → Differences = [16, 16, 16] → UIDloc = -12 (jumpy)

3. UIDglobNorm (Normalized global)
Same as UIDglob but divides by the mean. Make it comparable across sentences of diff lengths

4. UIDlocNorm (Normalized Local)
Same as UIDloc but divides by mean squared. Also for compatibility.

5. UID loc Prev Norm

compares each word with previous word (not mean). Very local measure

Feature Extraction

combines all our measures into feature vector for each sentence.

```
def extract_features (sent, lex_model, syn_model)
    feats = {}

  # get word sequence
    words = sent.get_word_sequence()
  # compute surprisals
    lex_surp = lex_model.compute_surprisal (words)
    syn_surp = syn_model.compute_surprisal (sent)
  # store totals
    feats ['lex_sum'] = sum (lex_surp)
    feats ['syn_sum'] = sum (syn_surp)

  # compute UID measures
    lex_uid = compute_all_uid (lex_surp)
    syn_uid = compute_all_uid (syn_surp)

  # store all UID measures
    for k,v in lex_uid.items():
        feats [f'lex_{k}'] = v
    for k, v in syn_uid.items():
        feats [f 'syn_{k}'] = v
    return feats
```

Pairwise Classification

```
def create_pairs (corp, var):
    x, y = [], []

  # First ordering: corpus first
    for c,v in zip (corp, var):
        x.append (make_diff (c, v)) # variant - corpus
        y.append (0)

  # second ordering: variant first
    for c,v in zip (corp, var):
        x.append (make_diff (v, c) # ~~variant~~ corpus - variant
        y.append (1)
    return np.array (x), np.array (y)
```

Running experiment

Experiment	Features	What we learn
Lexical surprisal	lex_sum	Baseline: how well does surprisal alone work?
UIDglob (lex)	lex_UIDglob	Does global UID help?
UID loc (lex)	lex_UID loc	Does local UID help?
Lex + UIDglob	lex_sum, lex_UIDglob	Does UID add to surprisal?
Lex + UIDloc	lex_sum, lex_UIDloc	Does local UID add to surprisal?

```
def run_experiments (corp_feat, var_feat, feat_names):
    # Extract only features we want to test
    corp = [{k: f[k] for k in feat_names} for f in corp_feat]
    var = [{k: f[k] for k in feat_names} for f in var_feat]

    # create pairs
    x,y = create_pairs (corp, var)

    # cross-validate
    acc, weights = cross_validate (x,y)

    return acc, weights
```

Run correlation analysis

check if UID measure is same as surprisal

```
run_uid_pipeline (train_file, test_file, max_test=50, max_var=10)
    train = parse_conllu (train_file)
    test = parse_conllu (test_file, max_test)

    Train Models

    lex_model = TrigramModel()
    lex_model.train([s.get_word_sequence() for s in train])

    syn_model = SyntacticModel()
    syn_model.train(train)

    for sent in test:
        variants = generate_variants (sent, max_var)
        for var in variants:
            corpus_features.append(extract_features(sent, lex_model, syn_model))
            variant_features.append(extract_features(var, lex_model, syn_model))
    all_experiments (corpus_features, variant_features)
    correlation_analysis (corpus_features) (in)
```